

Eight Runtime Behaviors

What the TypeScript runtime does that the syntax does not show — one model, one example, one review question each

October 2026

Why this sheet.

A “behavior” is a rule the runtime follows that you cannot see on the line itself. Promises are the biggest (they have their own sheet). These are the other seven, plus one more on equality. Each has: the one-line model to hold in your head, a concrete example with the value you end up holding, and the review question it raises.

1. Reference behavior

Model. A variable is a label. Objects, arrays, `Map`, `Set`, class instances are shared by label; numbers, strings, booleans, `null`, `undefined` are copied. `const` locks the label, never the contents.

```
const a = { n: 1 }; const b = a; b.n = 2; // a.n is 2: one object, two labels
let x = 1; let y = x; y = 2; // x is 1: primitives copy
const arr = [3, 1]; arr.sort(); // legal under const: contents changed, label did
not arr = []; // TypeError: label re-pointed
```

Copies are shallow: `[...a]`, `{ ...o }`, `a.slice()`, `new Map(m)` copy one level; nested objects inside are still shared. `structuredClone` is deep. `filter` copies the array, not the objects in it.

Review question. For every `push` / `sort` / `splice` / `o.x = v` / `delete`: who else is holding this object? A parameter, a shared module variable, a cached value, a value already returned to a caller.

2. Coercion behavior

Model.

Every operator has a preferred type and bends its operands to get it, silently. Python would raise; TypeScript converts and continues.

Expression	Converts	Result
<code>5 == "5"</code>	string to number	<code>true</code>
<code>0 == ""</code>	both to number	<code>true</code>
<code>"5" * 2</code>	string to number	<code>10</code>
<code>"5" + 2</code>	number to string	<code>"52"</code>
<code>"0" + 1</code>	number to string	<code>"01"</code> (the <code>for...in</code> index trap)
<code>Number("abc")</code>		<code>NaN</code> , no throw
<code>Number("")</code>		<code>0</code>
<code>Number(null)</code>		<code>0</code> ; <code>Number(undefined)</code> is <code>NaN</code>
<code>if ([])</code>	object to boolean	truthy
<code>if ("0")</code>	string to boolean	truthy
<code>[] + {}</code>	both to string	<code>"[object Object]"</code>
<code>true + 1</code>	boolean to number	<code>2</code>

The eight falsy values are `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`; everything else is truthy. `===` never coerces.

Review question. For every `==`, `+` with a string nearby, `Number(...)` on input, and `if (x)` on something that can be `0`, `""` or `[]`: what does the conversion produce, and did the author mean it?

3. Execution-order behavior (the event loop)

Model. One thread. Code runs in this order, every time: (1) the current synchronous task to its end; (2) every queued *microtask* (`await` continuations, `.then` / `.catch` callbacks, `queueMicrotask`); (3) one *macrotask* (`setTimeout`, `setInterval`, I/O completion, a new request); then back to (2).

```

console.log(1);
setTimeout(() => console.log(2), 0);           // macrotask: later
Promise.resolve().then(() => console.log(3));    // microtask: after sync, before timers
console.log(4);
// prints 1, 4, 3, 2

```

Consequences:

Fact	Why it matters
An <code>async</code> function runs synchronously up to its first <code>await</code>	the code before the first <code>await</code> happens immediately, in the caller's turn
Every <code>await</code> is a yield point	another request can run between the lines above and below it; that is the read-await-write race
<code>setTimeout(fn, 0)</code> is never immediate	it runs after every pending microtask; <code>0</code> means "next macrotask", not "now"
Timer delays are minimums	<code>setTimeout(fn, 100)</code> fires at 100 ms or later, never earlier
A long synchronous loop blocks everything	no request, timer or <code>.then</code> runs until it finishes; <code>await</code> inside a hot loop yields
<code>setInterval</code> keeps the process alive	until <code>clearInterval</code> or <code>.unref()</code>
Two <code>await</code> s in a row are sequential	the second starts only when the first settles; <code>Promise.all</code> starts both at once

Five orderings to be able to predict:

```

// A                                // B
await a(); await b();               await Promise.all([a(), b()]);
// a then b, sequential             // a and b together, wait for both

// C                                // D
const p = a(); await b(); await p;  ids.forEach(async id => await save(id));
console.log("x");                   // "x" prints before any save finishes
// a and b together (a started first)

// E
async function f() { console.log("in"); await 0; console.log("after"); }
f(); console.log("out");             // in, out, after

```

Review question. For every shared variable: is it read before an `await` and written after? For every pair of independent `await`s: could they be `Promise.all`? For every `setTimeout(fn, 0)`: does the author think it is immediate?

4. Scope and `this` behavior

Model. `let` and `const` are block-scoped and get a fresh binding per loop iteration; `var` is function-scoped and shared. A closure captures the *variable*, not its value at the time. `this` in a regular function is whatever called it (or `undefined` in modules); an arrow has no `this` of its own and uses the one where it was written.

```

for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3, 3, 3: one shared i
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0, 1, 2: one i per iteration

class Btn {
  label = "ok";
  handle() { return this.label; }
}
const b = new Btn();
b.handle(); // "ok": called on b
const h = b.handle; h(); // TypeError: this is undefined; the method was detached
el.on("click", b.handle); // same bug, in callback form
el.on("click", () => b.handle()); // fixed: arrow calls it on b
el.on("click", b.handle.bind(b)); // fixed: bound

```

Also: a function declared with `function` is hoisted (usable before its line); `const f = () => {}` is not. Variables declared with `let/const` cannot be read before their line (temporal dead zone).

Review question. For every callback that references a loop variable: `let` or `var`? For every `obj.method` passed without calling it: does it use `this` inside?

5. Error-propagation behavior

Model. A thrown error unwinds the *synchronous call stack* to the nearest enclosing `try`. An `await` boundary breaks the stack: a rejected Promise re-enters a stack only at the line that `await`s it. `finally` runs on every exit. An error nobody catches crashes Node (unhandled exception or unhandled rejection) or logs in a browser.

```
function a() { throw new Error("x"); }
function b() { a(); }
try { b(); } catch (e) { /* caught: a -> b -> here is one synchronous stack */ }

async function c() { throw new Error("y"); }
try { c(); } catch (e) { /* NOT caught: c returned a rejected Promise; the stack ended */ }
try { await c(); } catch (e) { /* caught: the await re-threw it here */ }
```

Situation	Where the error goes
<code>throw</code> in sync code	nearest enclosing <code>try</code> up the call stack
<code>throw</code> in an <code>async</code> function	the returned Promise rejects
rejection, awaited inside a <code>try</code>	that <code>try</code> 's <code>catch</code>
rejection, never awaited	unhandled rejection; process exits in modern Node
<code>throw</code> inside a <code>setTimeout</code> callback	nobody; the <code>try</code> around <code>setTimeout</code> ended long ago
<code>throw</code> inside a <code>.then</code> callback	the chain rejects; caught by a later <code>.catch</code> or an <code>await</code>
<code>throw</code> inside an Express async handler (v4)	not caught by Express; request hangs unless <code>next(err)</code>
<code>JSON.parse</code> on bad input	throws synchronously, right there
<code>catch (e)</code>	<code>e</code> is <code>unknown</code> ; narrow with <code>instanceof Error</code>
<code>finally</code>	runs after <code>return</code> , <code>throw</code> , or normal exit; cleanup goes here

Review question. For every `throw` and every call that can reject: trace up to the `try` that will see it, or say “nobody”. For every `catch`: does it swallow, log-and-continue, or re-throw with `cause`?

6. Type-erasure behavior

Model. Every type annotation, `interface`, `type`, generic, `as`, `!`, `satisfies`, `readonly`, `private` disappears at compile time. Nothing checks at runtime. Types are a contract the compiler enforces among your own files; the boundary with the outside world (HTTP body, headers, query, files, env, `JSON.parse`, databases returning `any`) is untyped and must be validated by hand.

```
interface User { id: number; name: string }
const u = JSON.parse('{ "id": "42" }') as User; // compiles; u.id is the STRING "42"; name is
undefined                                     // TypeError at runtime; the type said it was
there

function f(x: number) { return x * 2 }
f("5" as any); // compiles; returns 10 by coercion, not by type
```

Written	Runtime effect
<code>x: number</code>	none
<code>x as T</code>	none; pure relabel
<code>x!</code>	none; pure claim
<code>interface</code> / <code>type</code>	none; erased
<code>readonly</code> , <code>private</code> , <code>protected</code>	none; <code>#field</code> is the one runtime-private form
<code>enum</code>	a real object (numeric by default)
<code>class</code>	a real constructor function
<code>typeof x</code> in a value position	real; returns a string
<code>instanceof</code>	real; checks the prototype chain
<code>zod</code> / <code>joi</code> / manual <code>typeof</code> checks	real; this is how you actually validate

Review question. For every value that crosses the boundary: where is the runtime check, or is a type annotation standing in for one? Every `as`, `!` and `any` is a place the answer can be “nowhere”.

7. Prototype behavior

Model. A plain object `{}` inherits from `Object.prototype`, so it already has keys you did not put there: `constructor`, `toString`, `hasOwnProperty`, `__proto__`. Lookups fall through to the prototype when the own key is missing. `Map` and `Object.create(null)` have no such inheritance.

```
const o: Record<string, number[]> = {};  
"constructor" in o           // true, inherited  
o["constructor"]            // a function, not undefined  
o[userKey].push(1)          // if userKey is "constructor": TypeError; if "__proto__":  
pollutes every object  
Object.keys(o)              // [] : own keys only; the inherited ones are hidden from  
iteration  
for (const k in o) {}       // can include inherited enumerable keys from a modified  
prototype
```

Review question. Is a plain object indexed by a user-controlled key? Then `Map`. Is `k in obj` used where `Object.hasOwn(obj, k)` or `map.has(k)` was meant?

8. Equality behavior

Model. Objects compare by identity, never by contents. Two separately built objects are never `===`, even with identical fields. Primitives compare by value. `NaN` is not equal to anything, including itself.

```
{ a: 1 } === { a: 1 }      // false: two objects  
[1, 2] === [1, 2]         // false  
new Date(0) === new Date(0) // false; compare .getTime()  
"a" === "a"              // true: primitives  
NaN === NaN              // false; use Number.isNaN  
const x = { a: 1 }; const y = x; x === y // true: same object
```

To compare	Use
two primitives	<code>===</code>
two dates	<code>d1.getTime() === d2.getTime()</code>
two arrays by contents	length + every element, or a helper
two objects by contents	field by field, <code>JSON.stringify</code> both (key-order sensitive), or a deep-equal helper
membership of an object in an array	<code>arr.includes(obj)</code> is identity; <code>arr.some((o) => o.id === id)</code> for contents
a <code>Set</code> or <code>Map</code> keyed by objects	keys are by identity; <code>set.has({ a: 1 })</code> is <code>false</code> for a fresh literal
<code>NaN</code>	<code>Number.isNaN(x)</code>

Review question. For every `===`, `includes`, `indexOf`, `Set.has`, `Map.get` on an object or date: is the author comparing identity when they meant contents?

The eight in one table

#	Behavior	One-line model	First place to look
1	Reference	labels share objects; copies are shallow; <code>const</code> locks the label	every mutation
2	Coercion	operators convert silently; <code>===</code> does not	<code>==</code> , <code>+</code> , <code>Number()</code> , <code>if (x)</code>
3	Execution order	sync, then all microtasks, then one macrotask; every <code>await</code> yields	shared state around <code>await</code> ; <code>setTimeout(fn, 0)</code>
4	Scope and <code>this</code>	<code>let</code> per iteration, <code>var</code> shared; <code>this</code> is the caller; arrows inherit	<code>var</code> in loops; detached methods
5	Error propagation	unwinds the sync stack; <code>await</code> is the only async re-entry	every <code>throw</code> , every <code>catch</code>
6	Type erasure	types vanish; the boundary is untyped	<code>as</code> , <code>!</code> , <code>any</code> , <code>JSON.parse</code> , <code>req.body</code>
7	Prototype	plain objects have inherited keys	user-controlled keys on <code>{}</code>
8	Equality	objects by identity, primitives by value	<code>===</code> , <code>includes</code> , <code>Set.has</code> on objects